

System Level Programming

Lab # 1

Introduction

During this course you will learn the programming principles and techniques that are required to develop applications to solve different practical problems. By the end of this course, you will be able to use useful and efficient software programs to solve basic computing problems. We will be using C++ as the programming language and Visual Studio dot Net as the IDE (Integrated Development Environment). You must have become familiar with the development environment and the basic structure of a C++ program. Today you will be practicing the programs which take input from the user, perform some computations, and show the result to the user. You will get to know different datatypes. You will also get familiar with the const keyword.

Objective

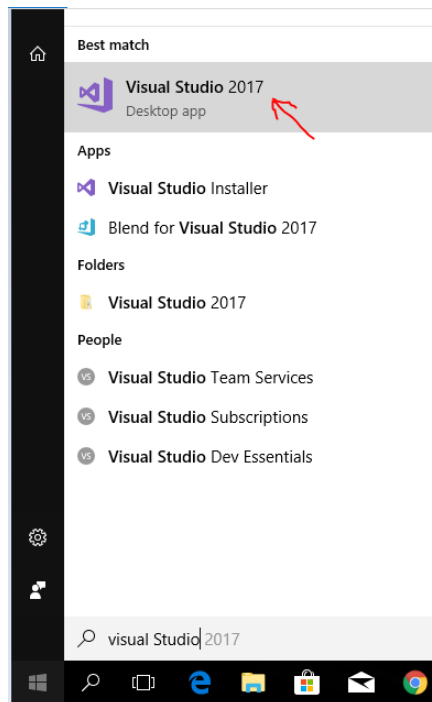
The first lab is intended to:

- a. Introduce the students to the development environment.
- b. Make the students write a basic program in C++.
- c. Getting Familiar with input output statements.
- d. Understanding several data types.
- e. Getting familiar with arithmetic operators and performing different calculations
- f. Getting familiar with the const keyword

Task 1 – Getting Familiar with the IDE

1. Click on the 'Start' button in the task bar, locate Visual Studio in the program list and launch it. (Figure 1)

Lab 1



Lab 1

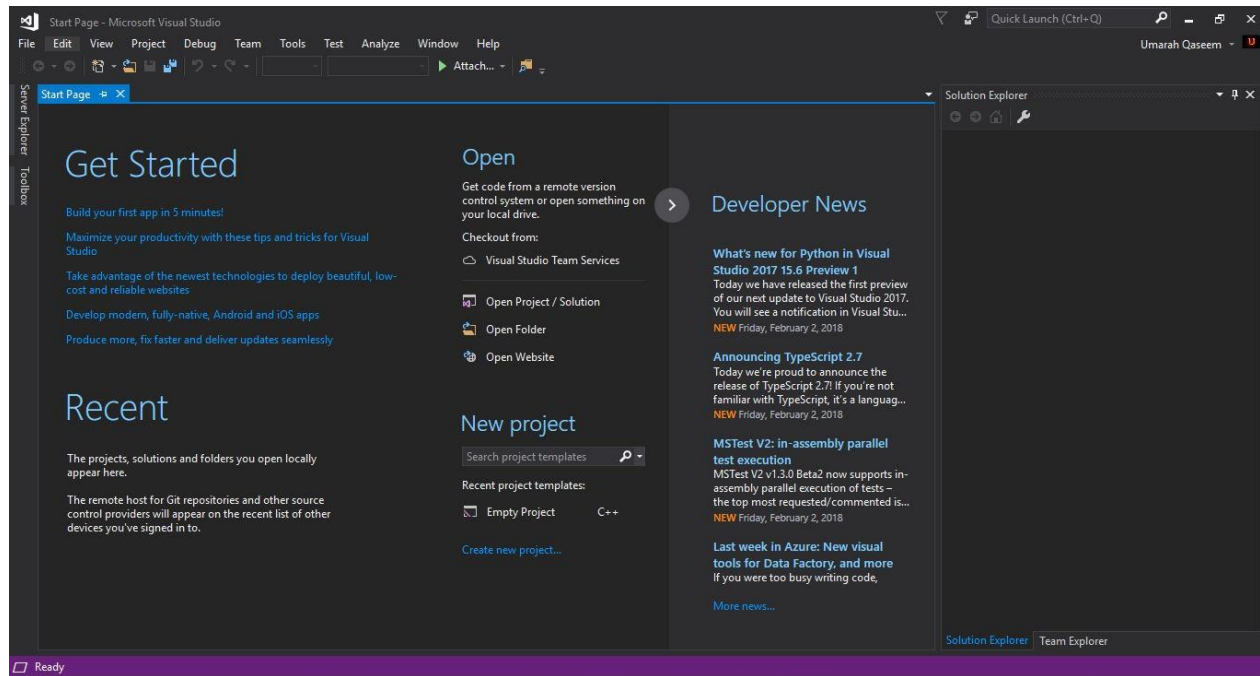


Figure 1 Visual Studio Environment

2. Click on 'File->New Project'. Select Visual C++ from the languages and 'Empty Project' from the templates as shown. Type a meaningful name and save it to a known location.

Lab 1

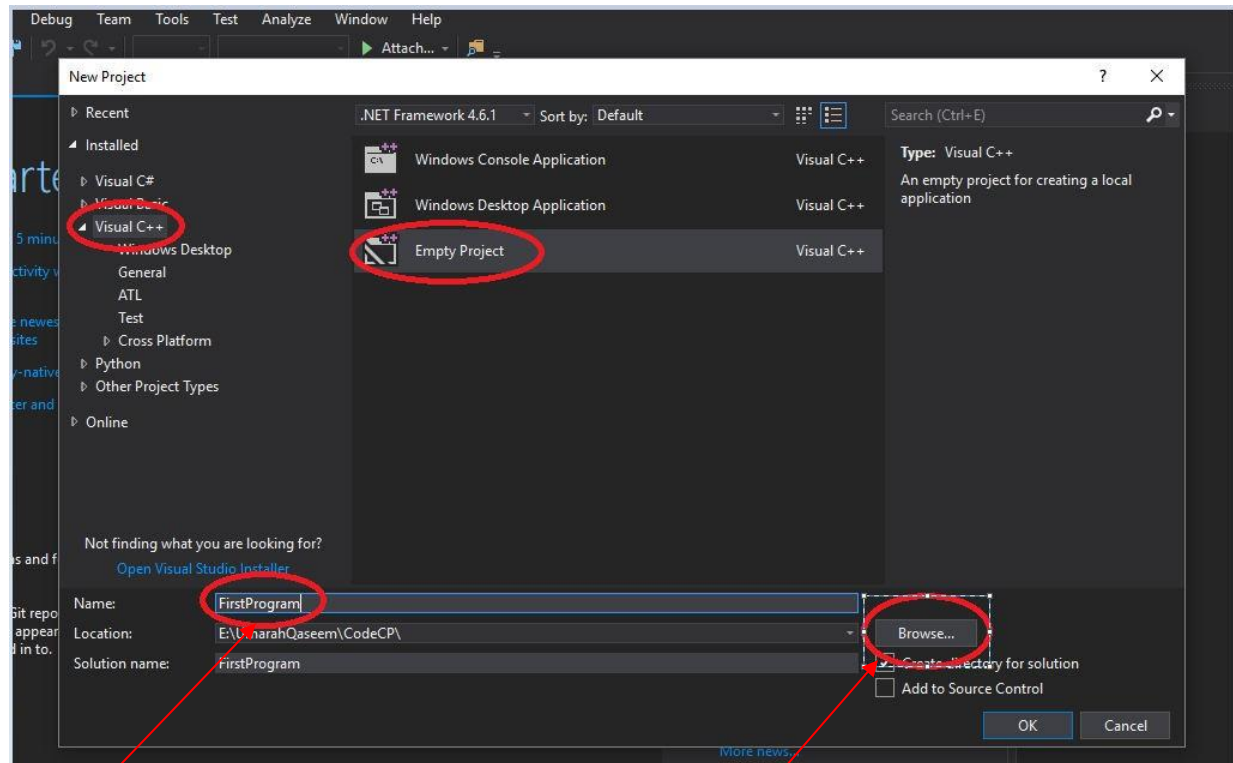


Figure 2 Creating a new project

Name of the project

Browse to change location of the project

Lab 1

- Click OK to create the project. Now the project is created. You will see solution explorer on the right of your screen. If it's not visible click View->Solution Explorer in the top bar. Now you can see 'Source files' in the solution explorer as circled in the figure below. Right click on it.

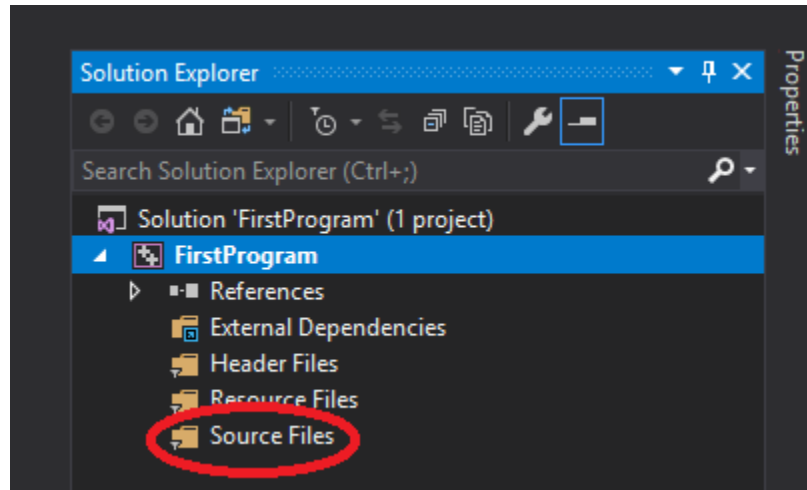
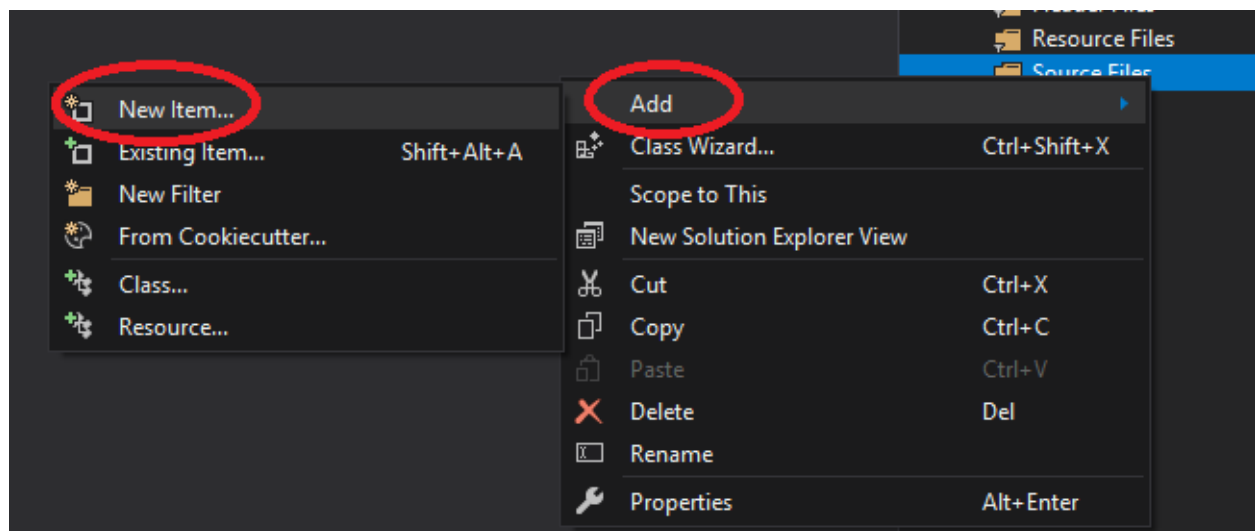


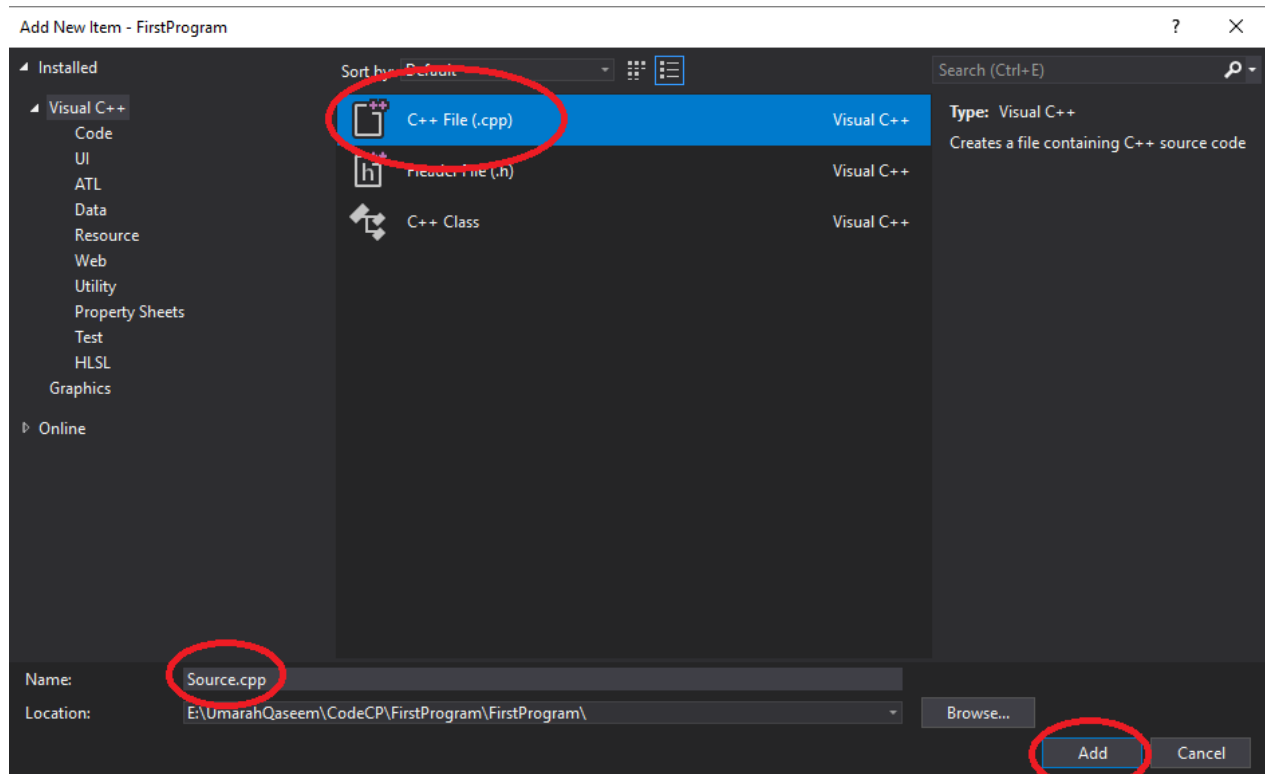
Figure 3 Project Created

- Right click on Source Files->Add->New Item.



- You will see this screen in the figure below for adding a new item. Select C++ File (.cpp). Give any meaningful name to the file. And click Add to create it.

Lab 1



6. Now your File is created. You can see the blank file on the screen. You will write your code in this file.

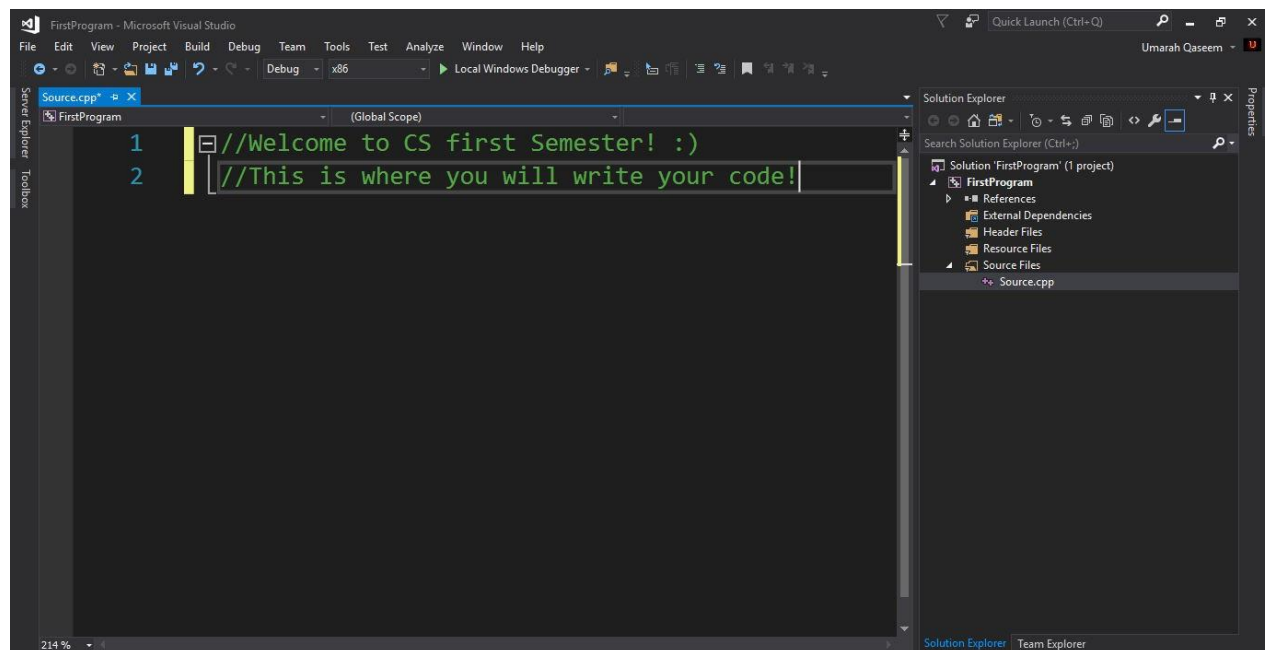
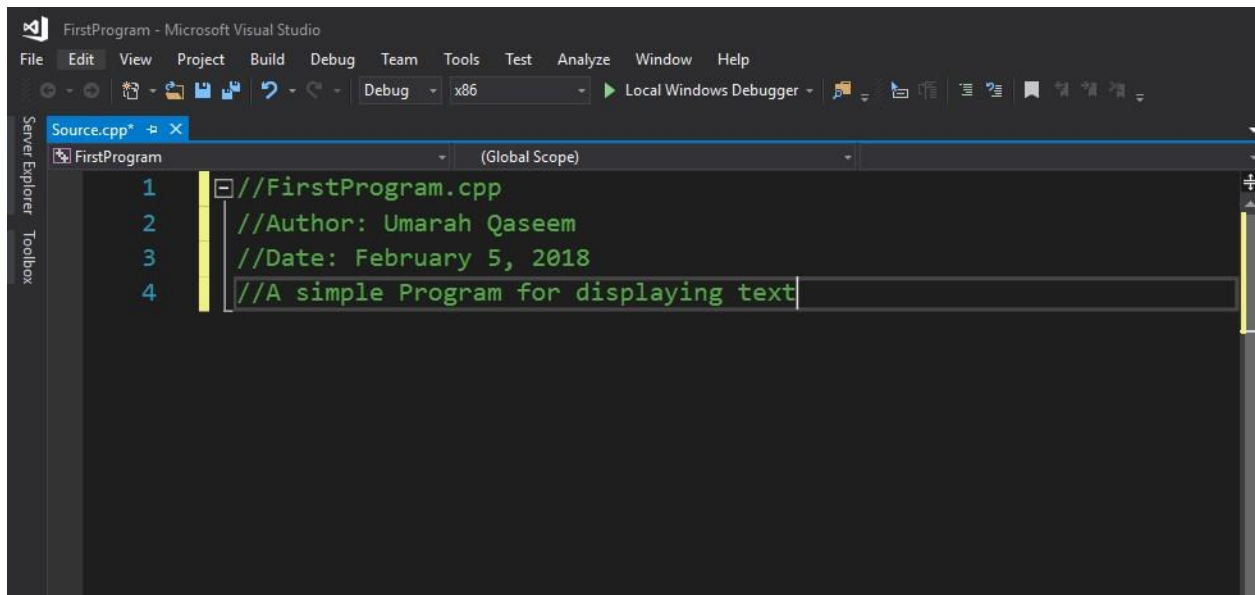


Figure 4 File created in the project

Lab 1

7. Now you will write a simple program that displays some text on the screen.
8. Start by writing meaningful comments on the top of the file. These should include:
 - a. Your name(s)
 - b. Date



- c. A brief description of the program

Figure 5 Comments at the start of the program

9. Complete your program as follows.

```
#include <iostream>
using namespace std;

int main()
{
    cout<<"My First program in C++";
    return 0;
}
```

10. Click on 'Build' and build your program.

Lab 1

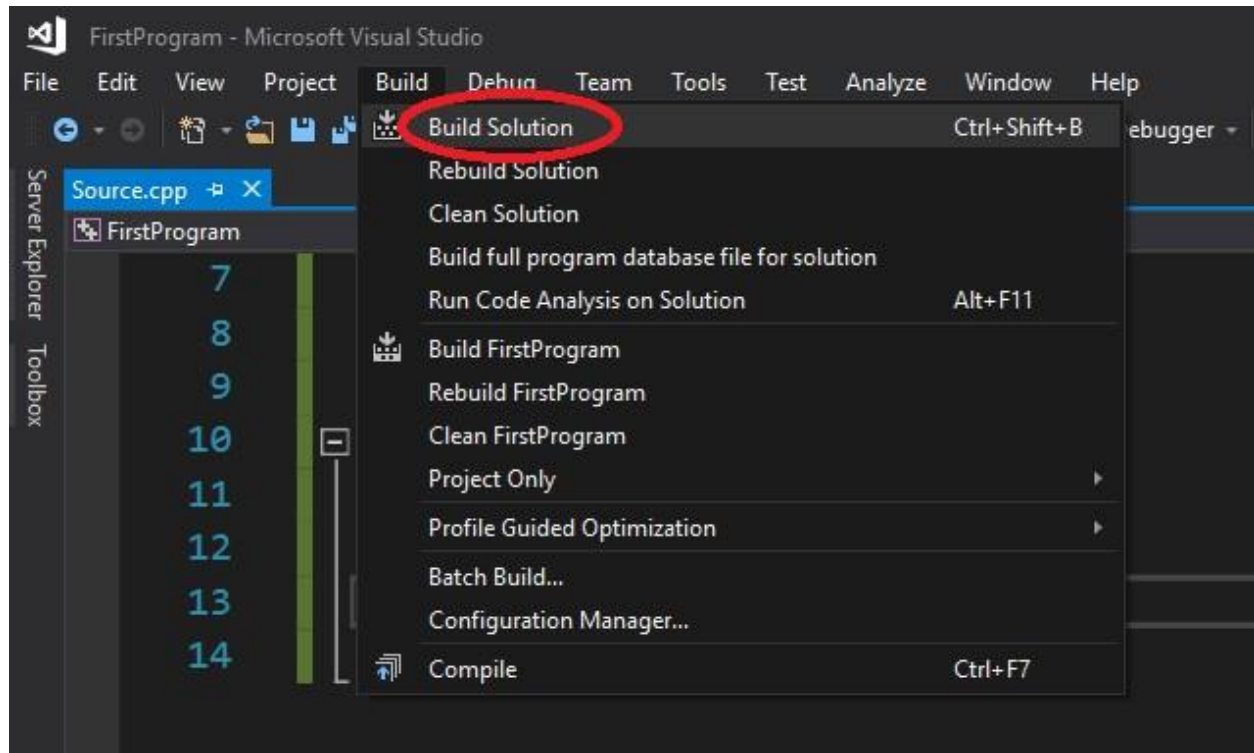


Figure 6 Building the program

11. If there are no errors, you will see a message 'Build Succeeded' at the bottom of the window. In case of errors, read them carefully and correct them.
12. After successfully building the program, execute it by selection 'Debug->Start Debugging'.

Lab 1

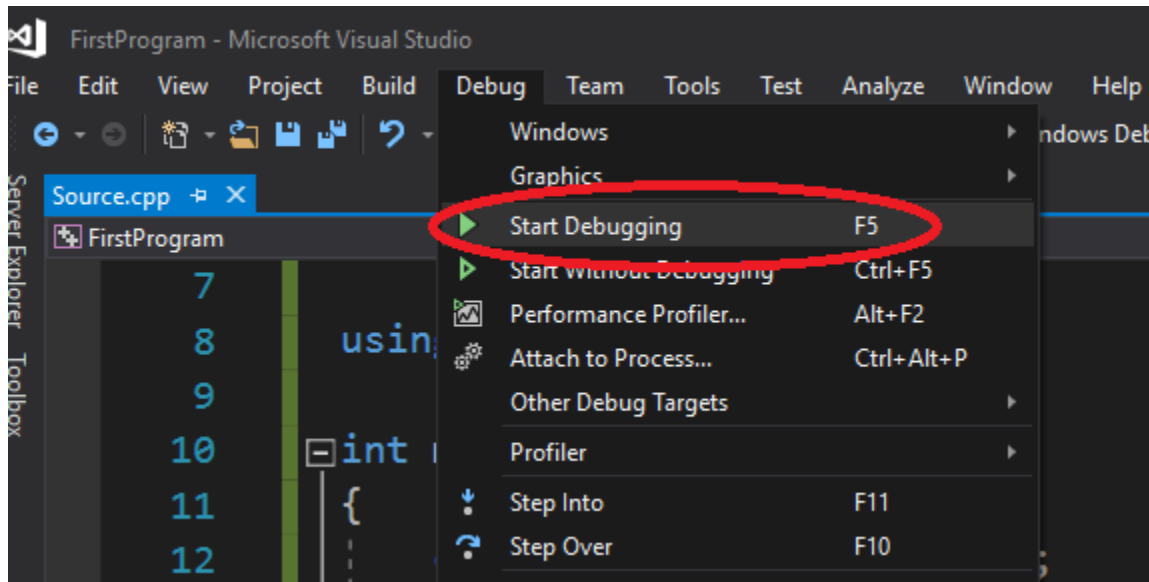


Figure 7 Executing the program

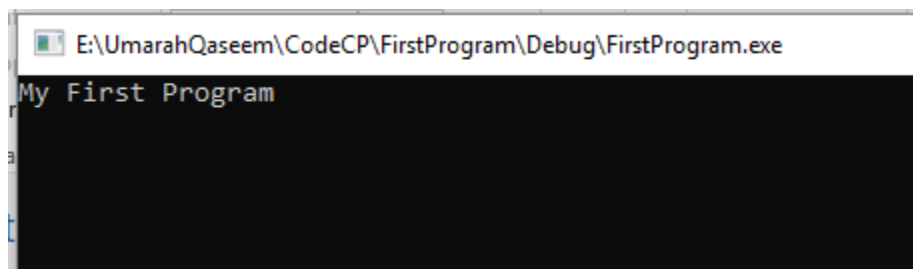
13. The output window will appear and disappear very quickly. In order to make the output window stay on the screen, modify the code as follows.

```
#include <iostream>
#include <conio.h>

using namespace std;

int main()
{
    cout<<"My First program in C++";
    _getch();
    return 0;
}
```

14. Execute the program and you will see an output like the following.



15. Now read the following details about the program you have just written.

Basic Structure of C++ Program

Comments

A comment is text that the compiler ignores but that is useful for programmers. Comments are normally used to annotate code for future reference. The compiler treats them as white space. You can use comments in testing to make certain lines of code inactive.

Comments start with a double slash symbol (//) and terminate at the end of the line. A comment can start at the beginning of the line or on the same line following a program statement.

There's a second comment style available in C++:

```
/*define char variable as character*/
```

This type of comment begins with the /* character pair and ends with */. You can write a multiline comment using the following two ways.

```
//define
```

```
//char variable
```

```
//as character
```

```
/*define
```

```
char variable
```

```
as character*/
```

Directives

The Preprocessor Directives

The lines #include <> are preprocessor directives. A preprocessor directive is an instruction to the *compiler*. The preprocessor directive #include tells the compiler to insert another file into your source file. A part of the compiler called the *preprocessor* deals with these directives before it begins the real compilation process. The file usually included by #include is called a *header file*.

The using Directive.

The line 'using namespace std' tells that all the program statements that follow are within the 'std' namespace. Various program components such as cout are defined within this namespace. If we didn't use this directive, we will need to append the name std to various program components like:

Lab 1

```
std::cout << "First program in C++";
```

Header Files

The preprocessor directive `#include` tells the compiler to add the header file `iostream` to the `FirstProgram.cpp` source file before compiling. A header file is added if the function/object defined in it is to be used in the program.

Main Function

When you run a C++ program, the first statement executed will be at the beginning of a function called `main()`. The program may consist of many functions and other program elements, but on startup, control always goes to `main()`. If there is no function called `main()` in your program, an error will be signaled.

Program Statements

The program statement is the fundamental unit of C++ programming. Each statement in C++ ends with a semicolon.

The statement: `cout << "First program in C++";`

causes the phrase in quotation marks to be displayed on the screen. The operator `<<` is called the *insertion* or *put to* operator. It directs the contents of the variable on its right to the object on its left.

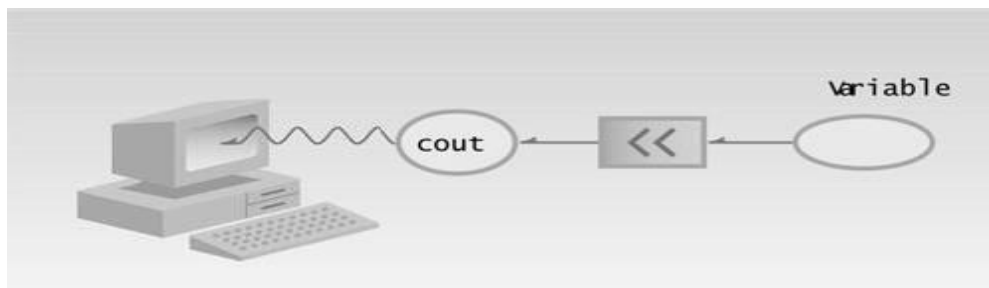


Figure 8 Stream insertion operator

Tokens

The smallest textual element that has some meanings to the code is called

token. Tokens: `main`, `{}`, `cout` etc.

Different types of tokens include:

1. Keywords.
2. Punctuators.
3. Identifiers.
4. Operators

Keywords:

Essential part of programming language is known as keywords. e.g. int , float , while , class etc. Some of the C++ keywords are:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Punctuators:

Symbols used for grouping or separating code are known as punctuators. Some examples are: “()”, “{}”, “[]”, comma, semicolon, colon etc.

Identifiers:

Program designed tokens used for naming variables, functions, classes, etc. are known as identifiers. e.g.x, abc, home etc.

Operators:

Operators are used in expressions to describe operations involving one or more operands. Examples include +, -, *, / etc.

Basic Input/Output

The example programs of the previous sections provided little interaction with the user, if any at all. They simply printed simple values on screen, but the standard library provides many additional ways to interact with the user via its input/output features. This section will present a short introduction to some of the most useful. C++ uses a convenient abstraction called *streams* to perform input and output operations in sequential media such as the screen, the keyboard, or a file. A *stream* is an entity where a program can either insert or extract characters to/from. There is no need to know details about the media associated with the stream or any of its internal specifications. All we need to know is that streams are a source/destination of characters, and that these characters are provided/accepted sequentially (i.e., one after another).

Lab 1

The standard library defines a handful of stream objects that can be used to access what are considered the standard sources and destinations of characters by the environment where the program runs:

stream	description
cin	standard input stream
cout	standard output stream

Lab 1

<code>cerr</code>	standard error (output) stream
<code>clog</code>	standard logging (output) stream

We are going to see in more detail only `cout` and `cin` (the standard output and input streams); `cerr` and `clog` are also output streams, so they essentially work like `cout`, with the only difference being that they identify streams for specific purposes: error messages and logging; which, in many cases, in most environment setups, they actually do the exact something: they print on screen, although they can also be individually redirected.

Standard output (`cout`)

On most program environments, the standard output by default is the screen, and the C++ streamobject defined to access it is `cout`.

For formatted output operations, `cout` is used together with the *insertion operator*, which is written as `<<` (i.e., two "less than" signs).

```
1 cout << "Output sentence"; // prints Output sentence on screen
2 cout << 120;               // prints number 120 on screen
3 cout << x;                 // prints the value of x on screen
```

The `<<` operator inserts the data that follows it into the stream that precedes it. In the examples above, it inserted the literal string `Output sentence`, the number `120`, and the value of variable `x` into the standard output stream `cout`. Notice that the sentence in the first statement is enclosed in double quotes (") because it is a string literal, while in the last one, `x` is not. The double quoting is what makes the difference; when the text is enclosed between them, the text is printed literally; when they are not, the text is interpreted as the identifier of a variable, and its value is printed instead. For example, these two sentences have very different results:

```
1 cout << "Hello"; // prints Hello
2 cout << Hello;   // prints the content of variable Hello
```

Multiple insertion operations (`<<`) may be chained in a single statement:

```
cout << "This " << " is a " << "single C++ statement";
```

This last statement would print the text `This is a single C++ statement`. Chaining insertions is especially useful to mix literals and variables in a single statement:

```
cout << "I am " << age << " years old and my zipcode is " << zipcode;
```

Assuming the `age` variable contains the value `24` and the `zipcode` variable contains `90064`, the output of the previous statement would be:

```
I am 24 years old and my zipcode is 90064
```

Lab 1

What `cout` does not do automatically is add line breaks at the end, unless instructed to do so. For example, take the following two statements inserting into `cout`:

```
cout << "This is a sentence.";
cout << "This is another sentence.";
```

The output would be in a single line, without any line breaks in between. Something like:

```
This is a sentence.This is another sentence.
```

To insert a line break, a new-line character shall be inserted at the exact position the line should be broken. In C++, a new-line character can be specified as `\n` (i.e., a backslash character followed by a lowercase `n`). For example:

```
1 cout << "First sentence.\n";
2 cout << "Second sentence.\nThird sentence.";
```

This produces the following output:

```
First sentence.
Second sentence.
Third sentence.
```

Alternatively, the `endl` manipulator can also be used to break lines. For example:

```
1 cout << "First sentence." << endl;
2 cout << "Second sentence." << endl;
```

This would print:

```
First sentence.
Second sentence.
```

The `endl` manipulator produces a newline character, exactly as the insertion of `'\n'` does; but it also has an additional behavior: the stream's buffer (if any) is flushed, which means that the output is requested to be physically written to the device, if it wasn't already. This affects mainly *fully buffered* streams, and `cout` is (generally) not a *fully buffered* stream. Still, it is generally a good idea to use `endl` only when flushing the stream would be a feature and `'\n'` when it would not. Bear in mind that a flushing operation incurs a certain overhead, and on some devices it may produce a delay.

Standard input (cin)

In most program environments, the standard input by default is the keyboard, and the C++ stream object defined to access it is `cin`.

Lab 1

For formatted input operations, `cin` is used together with the extraction operator, which is written as `>>` (i.e., two "greater than" signs). This operator is then followed by the variable where the extracted data is stored. For example:

```
1 int age;  
2 cin >> age;
```

The first statement declares a variable of type `int` called `age`, and the second extracts from `cin` a value to be stored in it. This operation makes the program wait for input from `cin`; generally, this means that the program will wait for the user to enter some sequence with the keyboard. In this case, note that the characters introduced using the keyboard are only transmitted to the program when the `ENTER` (or `RETURN`) key is pressed. Once the statement with the extraction operation on `cin` is reached, the program will wait for as long as needed until some input is introduced.

The extraction operation on `cin` uses the type of the variable after the `>>` operator to determine how it interprets the characters read from the input; if it is an integer, the format expected is a series of digits, if a string a sequence of characters, etc.

<pre>1 // i/o example 2 3 #include <iostream> 4 using namespace std; 5 6 int main () 7 { 8 int i; 9 cout << "Please enter an integer value: "; 10 cin >> i; 11 cout << "The value you entered is " << i; 12 cout << " and its double is " << i*2 << 13 ".\n"; 14 return 0; }</pre>	<pre>Please enter an integer value: 702 The value you entered is 702 and its double is 1404.</pre>
---	---

As you can see, extracting from `cin` seems to make the task of getting input from the standard input pretty simple and straightforward. But this method also has a big drawback. What happens in the example above if the user enters something else that cannot be interpreted as an integer? Well, in this case, the extraction operation fails. And this, by default, lets the program continue without setting a value for variable `i`, producing undetermined results if the value of `i` is used later.

This is very poor program behavior. Most programs are expected to behave in an expected manner no matter what the user types, handling invalid values appropriately. Only very simple programs should rely on values extracted directly from `cin` without further checking. A little later we will see how *stringstreams* can be used to have better control over user input. Extractions on `cin` can also be chained to request more than one datum in a single statement:

```
cin >> a >> b;
```


This is equivalent to:

```
1 cin >> a;  
2 cin >> b;
```

In both cases, the user is expected to introduce two values, one for variable `a`, and another for variable `b`. Any kind of space is used to separate two consecutive input operations; this may either be a space, a tab, or a new-line character.

Variables and types

The usefulness of the "Hello World" programs shown in the previous chapter is rather questionable. We had to write several lines of code, compile them, and then execute the resulting program, just to obtain the result of a simple sentence written on the screen. It certainly would have been much faster to type the output sentence ourselves.

However, programming is not limited only to printing simple texts on the screen. In order to go a little further on and to become able to write programs that perform useful tasks that really save us work, we need to introduce the concept of *variable*.

Let's imagine that I ask you to remember the number 5, and then I ask you to also memorize the number 2 at the same time. You have just stored two different values in your memory (5 and 2). Now, if I ask you to add 1 to the first number I said, you should be retaining the numbers 6 (that is 5+1) and 2 in your memory. Then we could, for example, subtract these values and obtain 4 as result.

The whole process described above is a simile of what a computer can do with two variables. The same process can be expressed in C++ with the following set of statements:

```
1 a = 5;  
2 b = 2;  
3 a = a + 1;  
4 result = a - b;
```

Obviously, this is a very simple example, since we have only used two small integer values, but consider that your computer can store millions of numbers like these at the same time and conduct sophisticated mathematical operations with them.

We can now define *variable* as a portion of memory to store a value.

Each variable needs a name that identifies it and distinguishes it from the others. For example, in the previous code the variable names were `a`, `b`, and `result`, but we could have called the variables any names we could have come up with, as long as they were valid C++ identifiers.

Identifiers

A *valid identifier* is a sequence of one or more letters, digits, or underscore characters (`_`). Spaces, punctuation marks, and symbols cannot be part of an identifier. In addition, identifiers shall always begin with a letter. They can also begin with an underline character (`_`), but such identifiers are -on most cases- considered reserved for compiler-specific keywords or external identifiers, as well as identifiers containing two successive underscore characters anywhere. Inno case can they begin with a digit.

C++ uses a number of keywords to identify operations and data descriptions; therefore, identifiers created by a programmer cannot match these keywords. The standard reserved keywords that cannot be used for programmer created identifiers are:

`alignas, alignof, and, and_eq, asm, auto, bitand, bitor, bool, break, case, catch, char, char16_t, char32_t, class, compl, const, constexpr, const_cast, continue, decltype, default, delete, do, double, dynamic_cast, else, enum, explicit, export, extern, false, float, for, friend, goto, if, inline, int, long, mutable, namespace, new, noexcept, not, not_eq, nullptr, operator, or, or_eq, private, protected, public, register, reinterpret_cast, return, short, signed, sizeof, static, static_assert, static_cast, struct, switch, template, this, thread_local, throw, true, try, typedef, typeid, typename, union, unsigned, using, virtual, void, volatile, wchar_t, while, xor, xor_eq`

Specific compilers may also have additional specific reserved keywords.

Very important: The C++ language is a "case sensitive" language. That means that an identifier written in capital letters is not equivalent to another one with the same name but written in small letters. Thus, for example, `theRESULT` variable is not the same as the `result` variable or the `Result` variable. These are three different identifiers identifying three different variables.

Fundamental data types

The values of variables are stored somewhere in an unspecified location in the computer memory as zeros and ones. Our program does not need to know the exact location where a variable is stored; it can simply refer to it by its name. What the program needs to be aware of is the kind of data stored in the variable. It's not the same to store a simple integer as it is to store a letter or a large floating-point number; even though they are all represented using zeros and ones, they are not interpreted in the same way, and in many cases, they don't occupy the same amount of memory.

Fundamental data types are basic types implemented directly by the language that represent the basic storage units supported natively by most systems. They can mainly be classified into:

- **Character types:** They can represent a single character, such as `'A'` or `'$'`. The most basic type is `char`, which is a one-byte character. Other types are also provided for wider characters.
- **Numerical integer types:** They can store a whole number value, such as `7` or `1024`. They exist in a variety of sizes, and can either be *signed* or *unsigned*, depending on whether they support negative values or not.

- **Floating-point types:** They can represent real values, such as 3.14 or 0.01, with different levels of precision, depending on which of the three floating-point types is used.
- **Boolean type:** The boolean type, known in C++ as `bool`, can only represent one of two states, `true` or `false`.

Here is the complete list of fundamental types in C++:

Group	Type names*	Notes on size / precision
Character types	<code>char</code>	Exactly one byte in size. At least 8 bits.
	<code>char16_t</code>	Not smaller than <code>char</code> . At least 16 bits.
	<code>char32_t</code>	Not smaller than <code>char16_t</code> . At least 32 bits.
	<code>wchar_t</code>	Can represent the largest supported character set.
Integer (signed) types	<code>signed char</code>	Same size as <code>char</code> . At least 8 bits.
	<code>signed short int</code>	Not smaller than <code>char</code> . At least 16 bits.
	<code>signed int</code>	Not smaller than <code>short</code> . At least 16 bits.
	<code>signed long int</code>	Not smaller than <code>int</code> . At least 32 bits.
	<code>signed long long int</code>	Not smaller than <code>long</code> . At least 64 bits.
Integer (unsigned) types	<code>unsigned char</code>	(same size as their signed counterparts)
	<code>unsigned short int</code>	
	<code>unsigned int</code>	
	<code>unsigned long int</code>	
	<code>unsigned long long int</code>	
Floating-point types	<code>float</code>	
	<code>double</code>	Precision not less than <code>float</code>
	<code>long double</code>	Precision not less than <code>double</code>
Boolean type	<code>bool</code>	
Void type	<code>void</code>	no storage
Null pointer	<code>decltype(nullptr)</code>	

* The names of certain integer types can be abbreviated without their `signed` and `int` components - only the part not in italics is required to identify the type, the part in italics is optional. I.e., `signed short int` can be abbreviated as `signed short`, `short int`, or simply `short`; they all identify the same fundamental type.

Within each of the groups above, the difference between types is only their size (i.e., how much they occupy in memory): the first type in each group is the smallest, and the last is the largest, with each type being at least as large as the one preceding it in the same group. Other than that, the types in a group have the same properties.

Note in the panel above that other than `char` (which has a size of exactly one byte), none of the fundamental types has a standard size specified (but a minimum size, at most). Therefore, the

Lab 1

type is not required (and in many cases is not) exactly this minimum size. This does not mean that these types are of an undetermined size, but that there is no standard size across all compilers and machines; each compiler implementation may specify the sizes for these types that fit the best the architecture where the program is going to run. This rather generic size specification for types gives the C++ language a lot of flexibility to be adapted to work optimally in all kinds of platforms, both present and future.

Type sizes above are expressed in bits; the more bits a type has, the more distinct values it can represent, but at the same time, also consumes more space in memory:

Size	Unique representable values	Notes
8-bit	256	$= 2^8$
16-bit	65 536	$= 2^{16}$
32-bit	4 294 967 296	$= 2^{32}$ (~4 billion)
64-bit	18 446 744 073 709 551 616	$= 2^{64}$ (~18 billion billion)

For integer types, having more representable values means that the range of values they can represent is greater; for example, a 16-bit unsigned integer would be able to represent 65536 distinct values in the range 0 to 65535, while its signed counterpart would be able to represent, on most cases, values between -32768 and 32767. Note that the range of positive values is approximately halved in signed types compared to unsigned types, due to the fact that one of the 16 bits is used for the sign; this is a relatively modest difference in range, and seldom justifies the use of unsigned types based purely on the range of positive values they can represent.

For floating-point types, the size affects their precision, by having more or less bits for their significant and exponent.

If the size or precision of the type is not a concern, then `char`, `int`, and `double` are typically selected to represent characters, integers, and floating-point values, respectively. The other types in their respective groups are only used in very particular cases.

The properties of fundamental types in a particular system and compiler implementation can be obtained by using the [numeric limits](#) classes (see standard header `<limits>`). If for some reason, types of specific sizes are needed, the library defines certain fixed-size type aliases in header [<cstdint>](#).

The types described above (characters, integers, floating-point, and boolean) are collectively known as arithmetic types. But two additional fundamental types exist: `void`, which identifies the lack of type; and the type `nullptr`, which is a special type of pointer. Both types will be discussed further in a coming chapter about pointers.

C++ supports a wide variety of types based on the fundamental types discussed above; these other types are known as *compound data types*, and are one of the main strengths of the C++

language. We will also see them in more detail in future chapters.

Declaration of variables

C++ is a strongly-typed language, and requires every variable to be declared with its type before its first use. This informs the compiler the size to reserve in memory for the variable and how to interpret its value. The syntax to declare a new variable in C++ is straightforward: we simply write the type followed by the variable name (i.e., its identifier). For example:

```
1 int a;  
2 float mynumber;
```

These are two valid declarations of variables. The first one declares a variable of type `int` with the identifier `a`. The second one declares a variable of type `float` with the identifier `mynumber`. Once declared, the variables `a` and `mynumber` can be used within the rest of their scope in the program. If declaring more than one variable of the same type, they can all be declared in a single statement by separating their identifiers with commas. For example:

```
int a, b, c;
```

This declares three variables (`a`, `b` and `c`), all of them of type `int`, and has exactly the same meaning as:

```
1 int a;  
2 int b;  
3 int c;
```

To see what variable declarations look like in action within a program, let's have a look at the entire C++ code of the example about your mental memory proposed at the beginning of this chapter:

Lab 1

```
1 // operating with variables
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     // declaring variables:
9     int a, b;
10    int result;
11
12    // process:
13    a = 5;
14    b = 2;
15    a = a + 1;
16    result = a - b;
17
18    // print out the result:
19    cout << result;
20
21    // terminate the program:
22    return 0;
23 }
```

4

Don't be worried if something else than the variable declarations themselves look a bit strange to you. Most of it will be explained in more detail in coming chapters.

Initialization of variables

When the variables in the example above are declared, they have an undetermined value until they are assigned a value for the first time. But it is possible for a variable to have a specific value from the moment it is declared. This is called the *initialization* of the variable.

In C++, there are three ways to initialize variables. They are all equivalent and are reminiscent of the evolution of the language over the years:

The first one, known as *c-like initialization* (because it is inherited from the C language), consists of appending an equal sign followed by the value to which the variable is initialized:

```
type identifier = initial_value;
```

For example, to declare a variable of type `int` called `x` and initialize it to a value of zero from the same moment it is declared, we can write:

```
int x = 0;
```

A second method, known as *constructor initialization* (introduced by the C++ language), encloses the initial value between parentheses `( )`:

Lab 1

type identifier (initial_value);

For example:

```
int x (0);
```

Finally, a third method, known as *uniform initialization*, like the above, but using curly braces ({}), instead of parentheses (this was introduced by the revision of the C++ standard, in 2011):

type identifier {initial_value};

For example:

```
int x {0};
```

All three ways of initializing variables are valid and equivalent in C++.

```
1 // initialization of variables
2
3 #include <iostream>
4 using namespace std;
5
6 int main ()
7 {
8     int a=5;           // initial
9     value: 5
10    int b(3);          // initial
11    value: 3
12    int c{2};          // initial
13    value: 2
14    int result;        // initial
15    value undetermined
16
17    a = a + b;
18    result = a - c;
19    cout << result;
20
21    return 0;
22 }
```

6

Typed constant expressions.

Sometimes, it is just convenient to give a name to a constant value:

```
1 const double pi = 3.1415926;
2 const char tab = '\t';
```

We can then use these names instead of the literals they were defined to:

Lab 1

```
1  #include <iostream>
2  using namespace std;
3
4  const double pi = 3.14159;
5  const char newline = '\n';
6
7  int main ()
8  {
9      double r=5.0;           //
10     radius
11     double circle;
12
13     circle = 2 * pi * r;
14     cout << circle;
15     cout << newline;
16 }
```

31.4159

Operators

Once introduced to variables and constants, we can begin to operate with them by using *operators*. What follows is a complete list of operators. At this point, it is likely not necessary to know all of them, but they are all listed here to also serve as reference.

Assignment operator (=)

The assignment operator assigns a value to a variable.

```
x = 5;
```

This statement assigns the integer value 5 to the variable `x`. The assignment operation always takes place from right to left, and never the other way around:

```
x = y;
```

This statement assigns to variable `x` the value contained in variable `y`. The value of `x` at the moment this statement is executed is lost and replaced by the value of `y`.

Consider also that we are only assigning the value of `y` to `x` at the moment of the assignment operation. Therefore, if `y` changes at a later moment, it will not affect the new value taken by `x`.

For example, let's have a look at the following code - I have included the evolution of the content stored in the variables as comments:

Lab 1

```
1 // assignment operator
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7     int a, b;           // a:?,    b:?
8     a = 10;             // a:10, b:?
9     b = 4;              // a:10, b:4
10    a = b;              // a:4,    b:4
11    b = 7;              // a:4,    b:7
12
13    cout << "a:";
14    cout << a;
15    cout << " b:";
16    cout << b;
17 }
```

This program prints on screen the final values of `a` and `b` (4 and 7, respectively). Notice how `a` was not affected by the final modification of `b`, even though we declared `a = b` earlier.

Assignment operations are expressions that can be evaluated. That means that the assignment itself has a value, and -for fundamental types- this value is the one assigned in the operation. For example:

```
y = 2 + (x = 5);
```

In this expression, `y` is assigned the result of adding 2 and the value of another assignment expression (which has itself a value of 5). It is roughly equivalent to:

```
1 x = 5;
2 y = 2 + x;
```

With the final result of assigning 7 to `y`.

The following expression is also valid in C++:

```
x = y = z = 5;
```

It assigns 5 to the all three variables: `x`, `y` and `z`; always from right-to-left.

Arithmetic operators (+, -, *, /, %)

The five arithmetical operations supported by C++ are:

operator	description
+	addition

Lab 1

-	subtraction
*	multiplication
/	division
%	modulo

Operations of addition, subtraction, multiplication and division correspond literally to their respective mathematical operators. The last one, *modulo operator*, represented by a percentage sign (%), gives the remainder of a division of two values. For example:

```
x = 11 % 3;
```

results in variable `x` containing the value 2, since dividing 11 by 3 results in 3, with a remainder of 2.

+++++